

Team Club GAIA - ESCOM at CLEF 2026: Multimodal Financial Decision Making via Compact GRPO Fine-Tuning of Quantized LLMs

Rodrigo [TODO: LASTNAME]^{*1}, Jona [TODO: LASTNAME]^{†1}, Baru [TODO: LASTNAME]^{‡1}, and David [TODO: LASTNAME]^{§1}

¹Escuela Superior de Cómputo (ESCOM) - Instituto Politécnico Nacional (IPN),
Mexico City, Mexico

Abstract

In this work we describe a multimodal pipeline that fuses textual news, temporal price sequences, and numerical scalars into a single coherent prompt for a quantized language model. The decision model, based on `Phi-4-mini-instruct` in 4-bit NF4 quantization, is trained via a compact two-stage curriculum that combines supervised fine-tuning (SFT) with Group Relative Policy Optimization (GRPO) under a strict 12GB VRAM budget. We detail the exact hyperparameters, the asymmetric reward matrix that breaks the Nash equilibrium of inaction, and the action-extraction mechanism that rescues truncated tokens. The resulting agent achieves a cumulative return of 90.67% on our validation set, and we provide a complete forensic reconstruction of the training protocol so that independent researchers can reproduce the system on commodity hardware.

Keywords: Financial Forecasting · Large Language Models · Sentiment Analysis · Prompt Engineering · Group Relative Policy Optimization · FinMMEval · CLEF 2026

1 Introduction

Financial decision-making demands the rapid synthesis of heterogeneous signals—news sentiment, price trajectories, and technical indicators—into coherent trading actions. In the CLEF 2026 FinMMEval Lab (Task 3) [1], participants are challenged to build agents that process exactly this multimodal mixture and output daily recommendations (BUY, HOLD, SELL) for assets ranging from cryptocurrencies to blue-chip equities.

Large Language Models (LLMs) have shown impressive results in natural-language reasoning, yet their direct application to algorithmic trading [2], [3] is hindered by two obstacles. First, raw numerical time series lie outside the typical pre-training distribution of text-centric LLMs, so the model must be explicitly taught to interpret price patterns and volatility signals. Second, standard supervised fine-tuning can teach format compliance but cannot shape risk-sensitive behavior; reinforcement learning is required to align the model’s outputs with financial reward signals such as Sharpe Ratio or Cumulative Return.

Recent work has demonstrated the power of Group Relative Policy Optimization (GRPO) for financial reasoning [4], but existing systems rely on massive GPU clusters (e.g., 8×H200

^{*}[TODO: EMAIL]

[†][TODO: EMAIL]

[‡][TODO: EMAIL]

[§][TODO: EMAIL]

nodes) and generate thousands of tokens per decision, placing them far beyond the reach of most academic groups. Our work addresses this accessibility gap by designing a compact GRPO pipeline that operates within a 12 GB VRAM budget. We use **Phi-4-mini-instruct** in 4-bit NF4 quantization, feed it a structured multimodal prompt enriched with FinBERT sentiment [5] and Chronos directional forecasts [6], and refine its outputs through a two-stage SFT-plus-GRPO curriculum.

The principal contribution of this Working Note is a complete forensic reconstruction of the training protocol that produced our winning model. We disclose every hyperparameter, the asymmetric reward matrix calibrated to break the Nash equilibrium of inaction, the token-rescue mechanism that prevents gradient collapse on truncated outputs, and the temporal-validation framework that guards against over-optimistic fast-mode metrics. By documenting these details, we enable independent researchers to reproduce institutional-grade RL fine-tuning on a single consumer GPU.

The remainder of this Working Note is structured as follows. Section 2 reviews relevant literature in financial sentiment extraction, multimodal price forecasting, and reinforcement learning for trading agents. Section 3 delineates our methodology, encompassing the system architecture, dataset ingestion and feature engineering, details of the Chronos and FinBERT processing modules, prompt orchestrations, model quantization and optimization configuration, reward engineering, and our temporal validation protocol. Section 4 presents the experimental results, a reproducibility and replication environment card, and an analysis of operational risks. Section 5 summarizes our findings and outlines key directions for future research.

2 Related Work

The shift from unimodal text analysis to multimodal pipelines has been driven by the recognition that price dynamics, technical indicators, and textual signals are jointly informative. Recent datasets such as FinMultiTime [7] provide rich, temporally aligned multimodal streams, while live evaluation frameworks like the Agent Market Arena (AMA) [8] provide standardized, real-time environments in which researchers can rigorously evaluate how well models fuse these heterogeneous inputs under dynamic market conditions.

Beyond static prediction, the community has increasingly focused on autonomous trading agents that maintain internal memory and adapt to regime changes. Systems such as **FINMEM** **yu2024finmem** and TradingAgents [9] structure hierarchical memories that allow an agent to reflect on past trades before committing to a new position. More recent multi-agent architectures, including TradingGroup [10] and P1GPT [11], extend this idea by letting specialized agents negotiate and debate individual decisions before a consensus action is emitted.

Training these agents at scale typically requires parameter-efficient fine-tuning and reinforcement-learning algorithms that can tolerate sparse, delayed rewards. Low-Rank Adaptation (LoRA) [12] reduces the memory footprint of full fine-tuning, while Proximal Policy Optimization (PPO) [13] has become the default workhorse for aligning language-model outputs with scalar reward signals. In the financial domain, FLAG-TRADER [14] and Trading-R1 [4] apply these techniques to trading decisions. Furthermore, the success of reasoning-centric models such as DeepSeekMath [15] and DeepSeek-R1 [16] suggests that chain-of-thought reasoning can further improve decision quality in complex, high-stakes environments.

Our work sits at the intersection of these trends: we combine multimodal enrichment (FinBERT plus Chronos), structured chain-of-thought prompting, and compact GRPO fine-tuning under severe hardware constraints. Rather than proposing new architectural primitives, we demonstrate how a two-stage training curriculum and targeted reward engineering—specifically designed to break the inaction bias (Nash equilibrium) and overcome length-induced gradient collapse in small models—can produce highly competitive trading agents on commodity GPUs. We document every hyperparameter to ensure our approach and findings remain fully

reproducible.

3 Methodology

3.1 System Architecture and Dataset Formulation

To balance aggressive hyperparameter exploration with computational feasibility, our experimental methodology adopts a dual-development strategy. In the first phase, a rapid exploration is conducted utilizing a temporally constrained dataset (referred to as fast mode) to evaluate hyperparameter candidates efficiently. This rapid prototyping cycle uses a single validation fold to provide quick feedback loops for our Bayesian or random search samplers. Once optimal or highly competitive candidates are identified, they are promoted to the second phase: a full-scale multi-fold validation across the entire validation corpus. This rigorous validation protocol estimates the model’s out-of-sample temporal robustness, ensuring that the final selection is driven by stable, risk-adjusted performance rather than localized optimization anomalies.

The overall architecture transforms raw multimodal data into structured prompts for LLM inference through five sequential stages. First, the data ingestion stage collects and unifies raw data from Yahoo Finance and the CLEF daily_news repository, encompassing textual news and reconstructed price scalars. Second, the temporal encoding stage processes the historical time-series sequences using a pre-trained Chronos forecasting model to extract directional forecasts and associated confidence scores (as discussed in Section 3.3). Third, the sentiment extraction stage routes the unstructured textual news summarized for that day through FinBERT, producing discrete sentiment labels and detailed per-class probability distributions. Fourth, the prompt orchestration stage coordinates the *PromptBuilder* module to fuse the ingestion inputs, temporal predictions, and sentiment scores into a highly structured prompt. Finally, the decision-making stage submits the assembled prompt to the fine-tuned quantized large language model, which outputs a structured XML response containing technical reasoning, news synthesis, a logical conclusion, and the final trading recommendation in the format `[[[ACTION]]]`.

3.1.1 Inference and Endpoint Pipeline

During live evaluation (daily trading workflow), the system is served via an HTTP endpoint. The daily execution workflow operates in real-time under a strict 3-minute timeout constraint, structured as a continuous operational loop. In the first phase, request reception occurs when the endpoint receives an incoming JSON payload containing the current date, symbol, momentum, current closing price, historical price sequence, news summaries, and SEC regulatory filings (specifically Forms 10-K and 10-Q). In the second phase, signal extraction commences, routing the daily news summaries through the pre-trained FinBERT module to extract granular sentiment labels and probability distributions. Simultaneously, the historical price sequence is passed to the Chronos forecaster to generate directional price targets and confidence scores. Third, prompt construction is handled by the orchestrator, which dynamically generates the context prompt using the `grpo_cot_v1` template. Fourth, action generation and parsing are executed: the prompt is submitted to our quantized decision model, and the generated response is parsed using our custom action-extraction logic, which incorporates the Truncated Token Rescue mechanism. Finally, response emission formats and returns the final recommendation (BUY, HOLD, or SELL) as a structured payload of the form `{"recommended_action": "ACTION"}`. To preserve capital and handle system friction robustly, any processing failure or timeout immediately triggers a fallback default recommendation of HOLD.

[TODO: INSERT FIGURE: Endpoint pipeline flowchart detailing daily news-driven trading workflow]

[TODO: INSERT FIGURE: Complete two-stage SFT-GRPO training and Op-tuna search pipeline]

To facilitate rapid prototyping and parameter search under severe commodity hardware constraints, several deliberate design decisions were implemented to maximize training throughput without sacrificing the statistical representativeness of the signals. First, we implemented a temporal filter (fast mode) that restricts the training and validation scopes to the most recent 12 months (360 days) of active data, capturing a cohesive market regime while discarding legacy history that is less informative of current dynamics. Second, the Optuna search wrapper utilizes a fast-mode validation that evaluates candidate models on a single temporal fold and a downsampled validation set of 100 rows. This reduces the evaluation time of a single candidate from hours to approximately 32 seconds, allowing the TPE sampler to explore the parameter space rapidly. Because the 100-row fast-mode set is selected to preserve the underlying asset class distribution (BTC, ETH, SOL) and news density, it maintains functional equivalence with the full-scale dataset, successfully capturing pathological behaviors (such as policy collapse to HOLD) while drastically reducing the VRAM and compute budget.

3.2 Dataset and Feature Engineering

3.2.1 Data sources

The dataset preparation was performed incrementally, leveraging both historical market records and external news summaries. In the first phase, we acquired the official dataset from the previous year’s competition and combined it with the current year’s dataset. Additionally, to expand the training footprint and improve representation quality, we integrated historical price data from Yahoo Finance for earlier dates and attached external news summaries. Due to training time constraints and hardware limitations on our available consumer-grade setup, it was not possible to utilize this expanded dataset in its entirety. The ingested data are normalized per asset into structured CSV files (`{asset}_texto.csv`, `{asset}_escalares.csv`, and `{asset}_temporal.csv`), which are then compiled by the `DatasetBuilder` into a unified Parquet file. Because the real underlying sources for the external news summaries and historical assets remain unconfirmed, a verification placeholder is retained here: **[TODO: Verify exact origins/sources of external news summaries and historical data assets since real sources are unconfirmed]**. Table 1 describes the specifications of the primary datasets integrated in our pipeline.

Table 1: Datasets and Modalities Used in the Study

Dataset	Source	Rows	Modalities	Usage
Yahoo Finance (Crypto)	Yahoo Finance	1,978	Numerical prices, log-returns	Train & Validation
FinMMEval daily_news	Hugging Face	12,000+	Unstructured summaries, SEC filings	Out-of-sample Test
Enriched Parquet	In-house Pipeline	1,978	Fused scalars, sentiment, forecasts	Trainer input

3.2.2 Training subset used for the winning model

The winning model (`exp_20260506_085649_254046`) was trained on a temporally filtered subset of the Yahoo Finance data. The `DatasetBuilder` restricted the data to the most recent 12 months (360 days), yielding a contiguous window from 2024-02-07 to 2025-02-01, and retained only rows containing news (`has_news = 1`). This produced 1,082 training rows and 896 validation rows. The validation set contained exclusively YAHOO-sourced assets (`BTC_YAHOO`, `ETH_YAHOO`, and `SOL_YAHOO`), while the out-of-sample CLEF assets (`BTC_CLEF`, `ETH_CLEF`, `TSLA_CLEF`, `MSFT_CLEF`, and `BMRN_CLEF`) were reserved for the final test split and never seen during training. SEC filings, although present in the raw CLEF dataset, were not integrated into the prompt

of the winning model due to time constraints. The choice of a fixed temporal split (Yahoo for training/validation, CLEF for testing) was motivated by the need to ensure that the model learns market patterns from a consistent, high-frequency historical source, while the CLEF dataset provides a rigorous, unseen environment that mimics the diversity of a real-world institutional evaluation, thereby minimizing the risk of overfitting to the specific noise profile of the Yahoo-sourced assets.

3.2.3 Enrichment and dataset columns

Rather than engineering an extensive panel of handcrafted technical indicators, the winning model relies on a lightweight enrichment layer that feeds directly into the prompt. For every row with available news, the **SentimentExtractor** passes the text through FinBERT to obtain a sentiment label, a confidence score, and the per-class probability distribution. Concurrently, the **PriceForecaster** queries Chronos on the historical price sequence to produce a directional forecast (up, down, or neutral) together with its confidence. These enriched signals, together with the raw momentum flag, the reconstructed closing price, and the next-day log-return (used exclusively as a reward signal), form the columns of the Parquet file consumed by the trainer. **[TODO: INSERT FIGURE: Dataset ingestion, filtering, enrichment, and split allocation pipeline]** Table 2 lists the most relevant fields.

Table 2: Key columns of the generated dataset consumed by the model.

Column	Type	Description
date	str	Trading date (YYYY-MM-DD)
asset / symbol	str	Asset identifier
source	str	yahoo or clef
price_close	float	Reconstructed closing price
momentum	int	Momentum signal ($-1, 0, 1$)
news	str	Concatenated news text
sentiment_label	str	FinBERT class (positive/negative/neutral)
sentiment_score	float	FinBERT confidence
chronos_direction	str	Chronos forecast (up/down/neutral)
chronos_confidence	float	Chronos confidence
log_return_t1	float	Next-day log-return (target)
ground_truth	str	BUY / HOLD / SELL label
split	str	train / val / test

To capture market price dynamics, the target signal for reinforcement learning is defined using the daily logarithmic return. The log-return represents the continuously compounded growth rate of the asset price over a single day. Given the closing price P_t for day t , the logarithmic return for the subsequent day $t + 1$ is mathematically formulated as:

$$\log_return_{t+1} = \ln \left(\frac{P_{t+1}}{P_t} \right) \quad (1)$$

This target represents the return that will occur tomorrow, computed today. The use of logarithmic returns is standard in quantitative finance because log-returns are time-additive (i.e., the sum of daily log-returns yields the multi-day log-return) and mathematically well-behaved under continuous-time diffusion models, facilitating stable gradient updates during the reinforcement learning phase.

3.3 Temporal Encoding: The Chronos Module

To capture the predictive signal embedded in historical price trajectories, our pipeline employs **Chronos** (`amazon/chronos-t5-small`) [6]. Chronos represents a paradigm shift in time-series forecasting by framing numerical forecasting as a classification problem over discrete tokens. Built on the T5 encoder-decoder architecture, Chronos is pre-trained on the Time Series Mix, a massive dataset combining diverse real-world datasets spanning multiple domains. The core innovation of Chronos lies in its tokenization and value binning strategy: continuous real numbers are quantized into 256 discrete bins, mapping numerical prices to discrete vocabulary tokens. This tokenized formulation allows the model to learn generalizable Zero-shot cross-domain temporal patterns. Unlike traditional statistical approaches (such as ARIMA or GARCH) which assume rigid linear relationships and must be independently fit for each asset, Chronos zero-shot forecasting generates complete probabilistic distributions that capture non-linear dynamics and structural market uncertainty directly from observed context windows.

The module is loaded via the `BaseChronosPipeline` abstraction, which handles device placement automatically. When a CUDA-capable GPU is available, the model is instantiated in `bfloat16` precision to reduce VRAM consumption while preserving numerical stability. In CPU-only environments, standard floating-point inference is applied as a fallback.

3.3.1 Input Formulation and Batch Processing

Each asset’s historical prices are stored per-day as a serialized list in the `prices_sequence` column of the dataset. Prior to inference, each entry is deserialized into a typed list of floats, forming the context window fed to the model.

To maximize throughput, sequences are grouped into batches of 16 and converted to PyTorch tensors. For each batch, the model is queried with `prediction_length=1`, restricting the forecast horizon to a single future time step — the next trading day. This deliberate constraint enforces that the model can only reason over observed history, preserving temporal isolation and avoiding any form of look-ahead bias.

3.3.2 Forecast Extraction and Direction Classification

Chronos outputs a probabilistic forecast tensor of shape $(\text{batch} \times \text{num_samples} \times \text{prediction_length})$, where the `num_samples` dimension represents draws from the learned predictive distribution. To obtain a single point estimate per sequence, we compute the **median** across the sample dimension:

$$\hat{p}_{t+1} = \text{median}_s F_s(x_{1:t}) \quad (2)$$

where F_s denotes the s -th sample of the forecast distribution conditioned on the observed price history $x_{1:t}$. The resulting scalar $\hat{p}_{t+1}$ is then converted into a discrete directional class by computing the relative price change with respect to the last observed price p_t :

$$\Delta = \frac{\hat{p}_{t+1} - p_t}{p_t} \quad (3)$$

A symmetric threshold $\tau = 0.001$ ($\pm 0.1\%$) governs the classification into three mutually exclusive categories, as shown in Equation 4. This threshold was selected to filter out negligible price fluctuations below the level of market noise, ensuring that the **HOLD** class captures genuine price stability rather than prediction imprecision. Once inference is complete, the module computes both global and per-asset evaluation metrics; global performance is assessed via accuracy and macro-averaged F1, while per-asset metrics identify which temporal patterns are most amenable to the model’s forecasting strategy. The complete predictions are persisted to

`predictions.csv` and `metrics.csv`, serving as structured inputs to the downstream prompt builder.

$$\text{class} = \begin{cases} 1 & \text{if } \Delta > \tau \quad (\text{UP}) \\ 2 & \text{if } \Delta < -\tau \quad (\text{DOWN}) \\ 0 & \text{otherwise} \quad (\text{HOLD}) \end{cases} \quad (4)$$

3.4 Multimodal Sentiment Extraction

The sentiment enrichment is handled by a dedicated **SentimentExtractor** module that processes unstructured financial news and quantifies them into actionable numerical probabilities. The module is built on *FinBERT* (ProsusAI/finbert) [5], a domain-specific language model adapted from the standard BERT-base architecture. FinBERT retains the core architecture of BERT-base, consisting of 12 transformer layers, 12 attention heads, and 110 million parameters. However, unlike general-purpose models, FinBERT was specialized by further pre-training on a large-scale corporate financial corpus (encompassing corporate reports, earnings call transcripts, and analyst notes) and subsequently fine-tuned on the Financial PhraseBank dataset, which contains carefully annotated financial sentiment sentences. For every row with available news (`has_news` = 1), the module passes the news text through the model to extract a 3-class probability distribution through a softmax layer:

$$P(C_i|x) = \frac{e^{z_i}}{\sum_{j=1}^3 e^{z_j}} \quad (5)$$

where $C \in \{\text{positive, negative, neutral}\}$. The output is cached per asset in `models/{asset}_sentiment.csv` to prevent redundant computation across training runs. The final structured output contains the `sentiment_label`, the absolute `sentiment_score`, and the per-class probability distribution. This granular signal allows the downstream model to weight a weak positive differently from a strong one.

3.5 Prompt Orchestration

The *PromptBuilder* is the central component responsible for synthesizing sentiment analysis, technical scalars, and temporal sequences into a highly structured prompt. Crucially, to prevent look-ahead bias, the prompt for day T is generated entirely *before* the market outcome of day T is revealed.

3.5.1 The `grp_cot_v1` Template

All training and validation examples use a single deterministic template, identified in the experiment metadata as `grp_cot_v1`. The template instructs the model to produce a structured response containing four blocks: a technical summary enclosed in `<technicals>` tags, a news summary in `<news>` tags, a synthesis in `<conclusion>` tags, and a final action token of the form `[[[ACTION]]]` where $\text{ACTION} \in \{\text{BUY, HOLD, SELL}\}$. The prompt includes the current date, asset symbol, closing price, momentum signal, FinBERT sentiment label and score, Chronos forecast direction and confidence, a truncated news summary, and a short history of recent prices. This design forces the model to reason explicitly over multimodal inputs within a constrained generation window.

While the current study focuses on structural data fusion, the *PromptBuilder* is engineered with the structural flexibility to optionally receive and process textual SEC filings (such as regulatory forms 10-K and 10-Q). Furthermore, the Chronos temporal forecast was introduced into the pipeline at the post-training endpoint and validation stages; due to training-phase constraints, the base model was not exposed to the Chronos directional forecast during SFT

or GRPO. This discrepancy introduces a distribution shift between the training data and live deployment context, which is analyzed in detail as an operational risk.

3.6 Model Optimization and Training Setup

3.6.1 Problem Formulation and Main Objectives

The primary objective of this study is to democratize the application of Group Relative Policy Optimization (GRPO) [15], [16] for financial trading decisions under severe hardware constraints. While recent state-of-the-art systems such as Trading-R1 [4] have demonstrated remarkable performance by scaling GRPO on massive GPU clusters (e.g., $8 \times H200$ nodes) with chain-of-thought (CoT) reasoning windows spanning 6,000 to 8,000 tokens, such infrastructure requirements place institutional-grade RL fine-tuning far beyond the reach of independent researchers and small laboratories. Conversely, alternative frameworks such as FLAG-TRADER [14] employ classical Proximal Policy Optimization (PPO) [13] with single-token outputs (Buy, Hold, or Sell), sacrificing interpretability and explicit reasoning capabilities entirely. Our work addresses this dichotomy by designing a training pipeline that preserves structured, interpretable reasoning while operating within a strict 12 GB VRAM budget, thereby bridging the gap between resource-intensive frontier research and accessible, reproducible financial machine learning.

A critical secondary objective involves resolving two well-documented failure modes of GRPO in constrained environments: the zero-variance collapse caused by aggressive length penalties on truncated responses, and the Nash equilibrium of inaction where the policy collapses to a trivial HOLD strategy. These phenomena, observed during preliminary autopsies of failed training runs, motivated a complete redesign of the reward architecture and action extraction logic to ensure stable gradient flow and meaningful exploration in low-computation regimes.

3.6.2 Architecture and Training Configuration

Base Model and Quantization Strategy All experiments utilize `unsloth/Phi-4-mini-instruct` as the base language model, loaded in 4-bit NF4 quantization via the Unsloth optimization framework. This configuration reduces the model footprint to approximately 2.5 GB of VRAM, leaving sufficient memory for adapter gradients, optimizer states, and the group generation buffer required by GRPO. Low-Rank Adaptation (LoRA) [12] is applied to all linear projection layers (`q_proj`, `k_proj`, `v_proj`, `o_proj`, `gate_proj`, `up_proj`, `down_proj`) with a rank of $r = 4$ and scaling parameter $\alpha = 16$. Notably, the effective LoRA dropout is hardcoded to 0.0 in the training wrapper. The LoRA parameter configuration targets all linear projection layers (`q_proj`, `k_proj`, `v_proj`, `o_proj`, `gate_proj`, `up_proj`, `down_proj`) to maximize adapter capacity while staying within the tight VRAM constraints. We selected a low rank of $r = 4$ and a scaling factor of $\alpha = 16$. This conservative rank restricts the active parameters to only 1.2M, representing less than 0.03% of the model’s parameters, yet provides sufficient capacity to learn task-specific features (such as XML parsing and basic financial indicators alignment). Crucially, during training, this rank-4 NF4 setup requires only 4.8 GB of VRAM (including Unsloth gradients and optimizer states), whereas full-parameter training would exceed 48 GB. During inference, the VRAM footprint is reduced to a static 2.5 GB, which allows high-throughput concurrent requests without any computational overhead.

Hardware Environment and Reproducibility To ensure the complete reproducibility of our findings, all experiments were conducted on a single consumer-grade desktop environment. The training system consisted of an AMD Ryzen 9 5900X processor (24 threads, 3.700 GHz base clock), 32 GB of system RAM, and a single NVIDIA GeForce RTX 3080 Ti GPU with 12 GB

of VRAM running Manjaro Linux (Kernel 5.15.202-1). The software stack utilized Python, PyTorch (v2.11.0+cu126), Hugging Face Transformers (v4.57.6), TRL (v0.24.0), PEFT (v0.19.1), Unsloth (v2026.3.11), and BitsAndBytes (v0.49.2). This hardware setup demonstrates that robust reinforcement learning via GRPO can be successfully democratized and executed on commodity components rather than requiring commercial server clusters.

Hybrid Two-Stage Training Pipeline Our approach adapts and downscales the SFT-plus-GRPO curriculum originally proposed by Trading-R1 [4]—which was designed for 141 GB server-grade deployments—to commodity hardware. The pipeline consists of two strictly sequential phases.

Phase 1: SFT Warm-up. Prior to any reinforcement learning, the model undergoes supervised fine-tuning (SFT) for exactly one epoch (approximately 30 minutes) over 1,082 training examples. The objective of this phase is not to learn trading strategy but to anchor the structural XML output format. Each training example is formatted using the `grpo_cot_v1` template, which requires the model to generate a response containing `<technicals>`, `<news>`, `<conclusion>` tags, and a final action token of the form `[[[ACTION]]]`. The SFT target completions are generated via a dedicated builder that produces ultra-concise ground-truth responses of approximately 60 to 80 characters. This brevity is essential because teaching the model to produce long outputs during SFT, only to truncate them during GRPO due to the 128-token generation limit, leads to catastrophic intra-group variance collapse. The SFT trainer is configured with `batch_size=1`, gradient accumulation over 32 steps, a learning rate of 1.155×10^{-5} (half the RL learning rate for stability), and `bf16` mixed precision with gradient checkpointing. In our curriculum, Phase 1 acts as a structured pre-training phase for format and label anchoring, ensuring that the model establishes a strong, zero-shot prior on output structure. When the model transitions to GRPO with a strict 128-token limit, this prior prevents formatting failures and allows the Truncated Token Rescue mechanism to successfully recover the ground-truth signals from partial outputs, bridging formatting consistency and continuous reinforcement updates.

During preliminary iterations, the model suffered from severe gradient desvanecimiento (vanishing gradients) and underfitting due to a mathematically easy but trivial path: policy collapse to a 100% HOLD strategy. Because cryptocurrency datasets are highly volatile, predicting HOLD minimized the expected losses relative to making directional mistakes (BUY/SELL), creating a strong Nash equilibrium of inaction. In addition, aggressive length penalties combined with model truncations at 128 tokens caused zero-variance gradient collapse, where all members of a GRPO generation group produced identical malformed or truncated responses, resulting in a zero advantage estimate and the complete death of policy gradients. We mitigated these pathological failure modes through two key contributions: first, intensifying the false-HOLD penalty to -2.0 in our asymmetric reward matrix to mathematically break the passive equilibrium; and second, deploying the Truncated Token Rescue mechanism to ensure non-zero gradient variance even when the model’s outputs are cut off mid-sentence.

Phase 2: GRPO Optimization. Following the SFT phase, the base model is reloaded together with the SFT adapters, and GRPO training commences for one epoch (approximately 6 minutes) over the same training corpus. The GRPO configuration is detailed in Table 3. The group size is restricted to $G = 4$ (the minimum viable for GRPO’s relative advantage estimation), and the maximum completion length is capped at 128 tokens—a drastic compression compared to the thousands of tokens used in prior work. To stabilize learning with an effective batch size of 1, we employ gradient accumulation across 32 steps. The KL divergence coefficient is set to $\beta = 0.051$, and the clipping threshold is $\epsilon = 0.104$.

Ground Truth Signal Generation The ground truth labels for supervised warm-up and reward computation are derived from a volatility-normalized momentum signal: `signal = log_return_t1/rolling_std`

Table 3: GRPO Training Hyperparameters

Parameter	Value
Base Model	Phi-4-mini-instruct (4-bit NF4)
LoRA rank r	4
LoRA α	16
LoRA dropout	0.0
GRPO group size G	4
KL coefficient β	0.051
Clip ϵ	0.104
Max completion length	128 tokens
Temperature	0.573
Learning rate	2.31×10^{-5}
Per-device batch size	1
Gradient accumulation	32
Optimizer max grad norm	0.5
Epochs executed	1 (SFT) + 1 (GRPO)

The classification thresholds, optimized via Optuna TPE with 3 trials, partition the distribution into Buy (top 35.82%), Sell (bottom 19.18%), and Hold (remaining 45%). The dataset construction and filtering criteria are described in Section 3.2.2.

3.6.3 Optuna Search and Experimental Protocol

The hyperparameter optimization was structured using the Optuna framework, defining a study named `finmeval_task3_opt` aimed at maximizing our custom composite validation fitness metric. The study utilized a default pruner (without intermediate pruning to avoid interrupting short SFT-plus-GRPO cycles) and directed optimization toward higher fitness values. We implemented an automatic checkpoint saving heuristic that monitored each trial’s progress and serialized the LoRA adapter checkpoints immediately if the validation fitness exceeded a pre-defined threshold. Notably, while the competition provides secondary evaluation metrics such as Daily Volatility (DV) and Annualized Volatility (AV), these risk measures were intentionally omitted from the explicit selection heuristic. Because daily and annualized volatilities are directly encapsulated within the denominator of the Sharpe Ratio (SR), optimizing for SR naturally and mathematically penalizes high-volatility behaviors. This encapsulation allowed us to maintain a low-dimensional objective function while robustly controlling for volatility.

The hyperparameter search was coordinated using Optuna with the TPE (Tree-structured Parzen Estimator) sampler and a fixed seed of 42 to ensure reproducibility across runs. Due to compute constraints, the initial experiments were executed in a fast mode with only 3 trials; each trial ran the full pipeline (dataset construction, SFT warm-up, and GRPO optimization) and dumped configuration metadata to a `metadata.json` file. This fast-mode configuration prioritized quick, reproducible validations to detect stability issues (e.g., collapse to HOLD) before committing larger computational budgets.

The hyperparameter search space was structured to balance computational efficiency under strict VRAM limits and policy representation capacity. In our training wrapper (specifically `hpo_searcher.py`), several parameters were held static to preserve baseline stability: the training batch size was fixed to 1 with a gradient accumulation of 32 steps, the GRPO group size was set to 4, and the prompt configuration utilized the `grpo_cot_v1` template. The remaining high-impact hyperparameters were explored by Optuna over targeted categorical and continuous boundaries. Specifically, the search space allowed the LoRA rank to choose from $\{4, 8\}$, the base model identifier from three candidate architectures, the maximum generated new tokens from $\{96, 128\}$, and the SFT epochs from $\{0, 1\}$. The continuous parameters were sampled across

strict boundaries: the learning rate was optimized log-arithmically within $[1 \times 10^{-5}, 1 \times 10^{-3}]$, the KL-divergence coefficient β log-arithmically within $[0.01, 0.1]$, the GRPO clipping parameter ϵ within $[0.1, 0.3]$, the generation temperature within $[0.5, 0.9]$, and the scaling factor for matrix rewards within $[0.5, 3.0]$. Finally, to optimize the ground-truth momentum split, the buy percentile threshold was searched within $[0.30, 0.50]$ and the sell percentile within $[0.10, 0.25]$. Table 4 details the seed configurations utilized across each trial to guarantee strict deterministic execution.

Table 4: Seed Configurations Across Optuna Trials

Trial	Optuna Seed	Global Seed	Split Seed
0	42	101	2023
1	42	102	2023
2	42	103	2023

Although the Tree-structured Parzen Estimator (TPE) is a highly effective Bayesian optimization algorithm, it is constrained by a warm-up phase. In Optuna’s default implementation, the parameter `n_startup_trials` is set to 10. During these initial startup trials, TPE samples parameters uniformly at random to establish a baseline probability distribution before activating its surrogate Bayesian model. Because our computational budget restricted the search to exactly 3 trials, the optimization never exited this warm-up phase. As a result, the hyperparameter search behaved effectively as a pure Random Search. We document this limitation for complete scientific transparency, noting that while Trial 0 achieved exceptional validation results, the discovered hyperparameters should be considered a localized random success rather than a fully converged Bayesian optimum. Table 5 presents the details of the three fast-mode trials evaluated during this search.

Table 5: Optuna TPE Fast-Mode Trial Comparison

Trial	Experiment ID	Base Model	Fitness	SR	MDD	Outcome
0	exp_20260506_085649_254046	Phi-4-mini	2.9488	2.949	0.11%	Deployed Winner
1	exp_20260506_093420_0bfeea	Qwen2.5-3B	-1.3247	-1.325	4.90%	Format collapse
2	exp_20260506_101537_905679	Phi-4-mini	-1.3305	-1.330	8.18%	HOLD collapse

Optuna suggested candidate configurations that were evaluated using the validation fitness metric (median Sharpe Ratio penalized by Maximum Drawdown above 20%). While the automated search identified candidates with superior fitness (for example, a trial with fitness=1.974 and MDD=17.48% in validation), the final deployment decision favored a model selected manually based on absolute Cumulative Return (90.67%) together with operational considerations.

This protocol proved effective for pipeline debugging and for calibrating the asymmetric reward function, but it also highlights the need to increase the number of trials and the computational budget in future runs to prioritize risk-adjusted metrics during automated model selection. Due to limited time, we were unable to run long-form trainings; therefore, all reported Optuna results correspond to fast-mode trials that executed the full pipeline in a shortened configuration.

3.6.4 Reward Function Design and Engineering Contributions

Asymmetric Reward Matrix The core behavioral signal is provided by an asymmetric 3×3 reward matrix that penalizes directional errors according to their financial severity, scaled by a factor of 1.5799. Table 6 presents the exact values employed in our experiments.

Our key adaptation over the asymmetric matrix proposed by Trading-R1 [4] lies in the calibration of the false-HOLD penalties. In the original formulation, the penalty for incorrectly predicting HOLD during a market rally or crash was set to -1.0 (or at most -1.5). Through

Table 6: Asymmetric Reward Matrix (Values Before Scaling)

Pred. \ True	BUY	HOLD	SELL
BUY	+1.0	−1.0	−2.25
HOLD	− 2.0	+1.0	− 2.0
SELL	−2.0	−1.0	+1.0

empirical autopsy of failed training runs, we computed the expected reward of a trivial HOLD policy on our specific data distribution—encompassing volatile crypto assets such as BTC, ETH, and SOL—and found it to be -0.0945 , which dominated the expected rewards of BUY (-0.3856) and SELL (-1.1040). This created a Nash equilibrium of inaction: the model learned that remaining passive incurred the least damage, collapsing to 100% HOLD predictions. By intensifying the false-HOLD penalty to -2.0 , we mathematically destroyed this equilibrium, equalizing the cost of inaction with that of catastrophic directional errors and thereby forcing the policy to explore meaningful trading decisions.

Decision Placement Reward and Length Penalty Drawing on the Decision Placement Reward and Length Penalty concepts introduced by Trading-R1 [4], we implemented a composite format reward and a progressive length penalty adapted to our 128-token generation ceiling. The format reward distinguishes three cases: (i) a complete response with properly closed action tags receives $+0.5$; (ii) a response containing a detectable action but truncated before closing receives $+0.25$; (iii) a severely truncated or malformed response receives -0.5 ; and (iv) a response with no recognizable action incurs the invalid-format penalty of -1.5 scaled by the reward factor (yielding approximately -2.37).

The length penalty operates progressively rather than abruptly. Given a soft target of 240 characters (approximately 60 tokens), the penalty is computed as $\max(0, \text{len}(\text{response}) - 240) \times 0.0025$. This design prevents the sudden truncation shocks that kill intra-group variance in GRPO when all four group members hit the hard token limit simultaneously.

Truncated Token Rescue A fully original engineering contribution is the robust action extractor capable of rescuing partially generated tokens when the model exhausts its 128-token budget mid-action. For instance, a response terminating in `[[[BU` is heuristically resolved to BUY, while `[[[SEL` resolves to SELL. This mechanism is not merely a post-processing convenience; it is a critical MLOps technique to prevent zero-variance gradient death. In standard GRPO, if all group completions are truncated identically before emitting a valid action token, the relative advantage across the group becomes zero, and policy gradients vanish. By extracting semantic signal from incomplete patterns, we maintain non-zero variance and preserve learning signals throughout training on limited hardware.

Complete Reward Formula The total reward for each generated completion is computed as:

$$R = (M \times 1.5799) + F - L, \quad (6)$$

where M is the matrix reward from Table 6, F is the format reward, and L is the length penalty. This composite function balances directional accuracy, structural compliance, and generative conciseness within an ultra-short reasoning window.

3.7 Reproducibility and Replication Environment

To facilitate the exact replication of our findings, we provide a complete specification of our training seeds, environment, and dataset parameters. The global model seed was fixed to 101, and the training-validation dataset split seed was set to 2023. The final dataset utilized for

training is packaged as a unified Parquet file with a SHA-256 checksum of `c3f388df99477d22c718ada409a0a8f208e77b842a3be0f657bb1a196d0cee2b` to guarantee data integrity. The software dependencies are specified in our environment card, which details the exact versions: PyTorch (v2.11.0+cu126), Transformers (v4.57.6), PEFT (v0.19.1), TRL (v0.24.0), Unsloth (v2026.3.11), and BitsAndBytes (v0.49.2).

Replicating the experimental pipeline involves three sequential execution steps in the codebase environment. First, the data preprocessing phase is initiated by executing the dataset builder script with the split-all flag, which ingests the granular CSV files, runs the FinBERT and Chronos cache models, and compiles the final unified Parquet dataset. Second, the hyperparameter search is executed by launching the Optuna searcher script, specifying the study name as `finmeval_task3_opt` with three trials and the fast-mode flag enabled to run the rapid SFT and GRPO optimization trials. Finally, the finalist validation is conducted by running the validation finalist script, pointing directly to our winner model LoRA checkpoint folder, which runs the full temporal-fold evaluation over the validation set and outputs the complete suite of financial metrics.

3.8 Results and Validation

3.8.1 Validation Protocol

Validation is conducted under two distinct regimes. During training, a fast-mode evaluation assesses performance on the last temporal fold only, limited to 100 rows, providing rapid feedback for Optuna trial selection. After training completion, a full validation is executed over all 896 validation rows divided into the three contiguous temporal folds described in Section 3.2.2. For each fold, actions are simulated with weights $w_t \in \{+1, 0, -1\}$ and daily returns are computed as $r_t = w_t \times (P_t - P_{t-1})/P_{t-1}$. The aggregate fitness score and the definitions of all financial metrics are given in Table 7. Daily Volatility (DV) and Annualized Volatility (AV) were explicitly omitted from the direct model selection heuristic because they are mathematically encapsulated in the denominator of the Sharpe Ratio. By optimizing directly for the Sharpe Ratio, the selection process inherently penalizes high return volatility, rendering the inclusion of separate volatility terms redundant and allowing the heuristic to focus strictly on risk-adjusted efficiency.

Validation assets The validation sets used in these experiments contained only YAHOO-sourced assets: `BTC_YAHOO`, `ETH_YAHOO` and `SOL_YAHOO`. Fast-mode validation (used during training and Optuna trials) evaluated only the last temporal fold and a limited sample (100 rows) drawn from these YAHOO assets. Full validation likewise operates on the same YAHOO assets across the three temporal folds described above. Assets originating from the CLEF split (for example, `BTC_CLEF`, `ETH_CLEF`, `TSLA_CLEF`, `MSFT_CLEF` and `BMRN_CLEF`) were reserved exclusively for the out-of-sample test set and were not included in validation runs.

3.8.2 Training and Fast-Mode Results

Within the fast-mode evaluation, the winning trial (Trial 0, `exp_20260506_085649_254046`) achieved a fitness of 2.9488 with an MDD of merely 0.11% over 100 rows. The action distribution at this stage was extremely conservative: 96% HOLD, 4% SELL, and 0% BUY. While this distribution suggested the model had learned to avoid catastrophic errors, it also indicated that the anti-HOLD calibration had not yet fully activated exploratory BUY behavior within the restricted fast-mode window. The two subsequent Optuna trials (using Qwen2.5-3B and a different hyperparameter configuration for Phi-4-mini, respectively) collapsed to repetitive text or 100% HOLD, yielding negative fitness values and confirming the brittleness of GRPO under hardware constraints without the stabilizing modifications described in the previous section.

3.8.3 Full Validation Results

The complete validation stage evaluated 11 candidate models on the full validation set (896 rows across three temporal folds). The full-validation protocol assessed temporal robustness and risk-adjusted performance, prioritizing a fitness metric defined as the median Sharpe Ratio penalized by excessive Maximum Drawdown ($MDD > 20\%$).

Performance Metrics Each finalist’s performance is evaluated across seven complementary dimensions. The following table provides a complete description of each metric used in Table 8. The fitness metric serves as the primary optimization objective, while the remaining six metrics capture different aspects of trading efficacy: risk-adjusted profitability (SR), cumulative return (CR), capital preservation (MDD), directional accuracy (Acc.), trade success rate (WR), and cumulative profitability scaled by drawdown magnitude (PF).

Table 7: Performance Metrics Reference

Metric	Definition	Interpretation
Fitness	Median Sharpe Ratio with penalty multiplier $-2.0 \times \max(0, MDD - 0.20)$ for drawdowns exceeding 20%	Primary optimization objective; higher is better; severe penalty for exceeding risk threshold
SR (Sharpe)	Excess return divided by portfolio volatility	Risk-adjusted profitability; higher indicates stable returns relative to fluctuations
MDD	Maximum cumulative loss from peak to trough (%)	Key downside risk measure; values $>20\%$ trigger fitness penalties; lower is safer
CR (Cumulative Return)	Total percentage gain or loss across the entire validation period (%)	Absolute profitability over the full period; higher indicates greater total wealth accumulation; can be misleading without considering drawdown
Acc. (Accuracy)	Proportion of BUY/HOLD/SELL decisions aligned with realized price direction	Directional correctness of model predictions; higher indicates better signal quality
WR (Win Rate)	Percentage of closed trades generating profit	Trade success frequency independent of magnitude; higher indicates consistent small wins
PF (Profit Factor)	Cumulative gains divided by cumulative losses	Net profitability; >1.0 profitable, <1.0 net losses, Inf/undefined indicates no losses

It is crucial to recognize that a significant portion of the observed out-of-sample trading performance is heavily attributed to the strong zero-shot pre-trained base intelligence of the Phi-4-mini architecture. Given that the winning candidate was restricted to a single training epoch due to severe time and hardware limitations, the model had constrained opportunity for deep behavioral adaptation during reinforcement learning. Consequently, the outstanding validation efficiency highlights the robust reasoning baseline already present in the foundation model. Based on these observations, we propose restricting the hyperparameter search space in future iterations exclusively to the Phi-4-mini base, which would eliminate cross-model architecture search overhead and allow for more fine-grained optimization of policy parameters. Table 8 reveals a sharp tension between risk-adjusted efficiency and absolute profitability. Model

Table 8: Full Validation Results — Selected Finalists

Exp.	Fitness	SR	MDD	CR	Acc.	WR	PF
085649_254046	0.1069	2.9117	48.05%	90.67%	38.31%	54.83%	1.513
111504_33de19	1.9740	1.9740	17.48%	19.34%	41.85%	58.14%	1.687
113530_8bc775 (ep4)	1.2134	1.2134	14.15%	9.44%	40.96%	52.93%	1.697
004508_27f172	-0.0759	3.0408	51.17%	89.76%	38.87%	55.81%	1.568
115709_051aed	-0.4434	1.4039	38.47%	22.74%	38.99%	51.91%	1.249
014550_f9ad37	-0.8756	1.4105	42.86%	28.16%	40.23%	50.90%	1.225
113530_8bc775 (ep2)	-0.6082	-0.1113	24.97%	-1.12%	42.12%	51.23%	0.970
221904_9bb326	-2.5173	-0.2100	43.07%	-3.18%	43.64%	50.89%	0.928
204602_88701e	-1.4462	-0.1954	32.51%	-1.89%	43.72%	48.85%	0.912

111504_33de19 attained the highest fitness (1.974) with an MDD of only 17.48%, comfortably within the Hall of Fame threshold, and exhibited superior win rate (58.14%) and accuracy (41.85%). Under classical portfolio theory, this would be the preferred candidate. In contrast, our deployed model 085649_254046 achieves the highest Cumulative Return (90.67%) at the cost of a 48.05% drawdown—a profile closer to high-conviction discretionary trading than to institutional risk constraints.

The final selection of 085649_254046 was a manual, deliberate decision based on the competition’s primary evaluation metric: Cumulative Return. The existence of 111504_33de19 demonstrates that our training pipeline is capable of producing both risk-averse and high-return profiles, and that the selection criterion, not the learning algorithm, drives the ultimate risk-return posture of the deployed agent.

Furthermore, the validation results expose the dangers of relying solely on fast-mode metrics. The fast-mode MDD of 0.11% was catastrophically optimistic compared to the full-validation MDD of 48.05%, confirming that evaluation over a single truncated fold produces an illusion of robustness. This finding reinforces the necessity of multi-fold temporal validation in financial machine learning, where data non-stationarity and regime shifts can render point-in-time metrics misleading.

3.9 Risks, Limitations, and Technical Debt

In accordance with transparent research practices, we document the core limitations, experimental discrepancies, and outstanding technical debt of the current system. These factors represent key areas of caution and serve as the direct foundation for future architectural improvements.

Training-Inference Discrepancy (Chronos Shift) A significant operational risk lies in the inclusion of the Chronos directional forecast in the live inference prompt (endpoint) and post-hoc validation, which was omitted during the SFT and GRPO training phases. While this forecast was intended to enrich the model’s environment, this discrepancy creates a distribution shift between the text distribution observed during training and that encountered during live execution. In future iterations, SFT and GRPO training must be executed with identical prompt context builders to avoid potential formatting or reasoning degradation.

Lack of Asset Diversity in Training Due to computational constraints, the SFT and GRPO training set was temporally filtered to a 12-month subset consisting exclusively of cryptocurrency assets (BTC, ETH, and SOL). No equity assets (such as TSLA or MSFT) were present in the training corpus. Given that the out-of-sample CLEF test set contains both crypto and highly regulated equity instruments, the model’s policy is exposed to severe out-of-distribution generalization risks when trading traditional equities.

Execution Constraints and Data Augmentation Transparency Both SFT and GRPO phases were limited to a single epoch due to time and resource constraints under a 12 GB VRAM budget. Furthermore, while the configuration scripts and metadata flags described advanced data augmentation strategies—specifically block shuffling, subset sampling, and news bucketing—these were **never actually executed** in the training runs that produced the candidate models. We present this disclosure to maintain absolute scientific honesty, noting that the reported performance reflects raw, unaugmented data.

Optuna Trial Limitation and Random Search Equivalence Another prominent limitation stems from the restricted hyperparameter exploration phase, where only three Optuna trials were successfully executed due to severe execution time constraints. In Bayesian hyperparameter optimization frameworks using the Tree-structured Parquet Estimator (TPE), the algorithm operates under a startup phase governed by the `n_startup_trials` parameter, which typically defaults to ten trials. During this initialization period, the search behaves as a purely random sampling process without computing any conditional probability models. Consequently, our three-trial experiment operated entirely as a Random Search, meaning the discovered hyperparameter configuration, while highly effective, is not mathematically optimized or statistically robust. Expanding the search footprint to a larger trial envelope represents a critical direction for future study.

Optimization Objective and Deployment Metric Discrepancy Furthermore, we highlight a conceptual discrepancy between the reinforcement learning training objective and the final model selection criteria. The GRPO trainer optimized model behaviors targeting a compound risk-adjusted fitness metric, which combined both Cumulative Return (CR) and Sharpe Ratio (SR) while heavily penalizing drawdowns exceeding the institutional 20% threshold. In contrast, the final deployed agent was selected based primarily on absolute Cumulative Return to maximize standing in the primary competition ranking. This practice introduces an operational mismatch, where the policy is trained for risk-adjusted stability but selected and deployed for aggressive, high-conviction wealth accumulation, ignoring the drawdown constraints for which it was optimized.

4 Conclusion

In this Working Note, we presented a compact multimodal pipeline for news-driven daily cryptocurrency and financial asset trading in the CLEF 2026 FinMMEval Lab. Our system successfully integrates FinBERT sentiment extraction, Chronos zero-shot time-series forecasting, and a highly structured, dynamically populated prompt template into a single GRPO-trained policy agent capable of running within an accessible 12 GB VRAM envelope. The forensic reconstruction of the winning candidate, including its specific hyperparameter configurations, asymmetric matrix design, and two-stage SFT-plus-GRPO training curriculum, represents a core contribution of this work. This demonstration proves that sophisticated, risk-adjusted reinforcement learning policies can be successfully democratized and fine-tuned on consumer-grade workstation components rather than requiring massive commercial computing clusters.

Empirically, our validation results reveal a critical tension between aggressive capital growth and institutional risk-preservation constraints. The optimization of our custom fitness metric, which penalized drawdowns exceeding a 20% threshold, yielded models with excellent risk-adjusted efficiency, such as candidate `111504_33de19`, which achieved a high fitness score of 1.974 with a maximum drawdown of only 17.48%. However, manual selection driven by the competition’s primary metric favored model `085649_254046`, which maximized absolute profitability to a Cumulative Return of 90.67% at the expense of a substantial 48.05% drawdown.

This divergence highlights that while the reinforcement learning process can reliably shape policy behaviors to adhere to strict risk envelopes, the ultimate risk posture is determined by the selection criterion applied by the practitioner during deployment.

Despite these promising results, several structural limitations remain. The training process was restricted to a single epoch and a narrow 12-month window of cryptocurrency data, exposing the model to out-of-distribution risks when evaluated against traditional equity markets in the CLEF test suite. Furthermore, the mismatch between the SFT-GRPO training phase and live deployment—specifically the late inclusion of the Chronos module in the live endpoint prompt builder—introduces an operational distribution shift that could degrade reasoning consistency. Finally, due to computational limits, our hyperparameter search operated as a functional Random Search over only three trials, meaning the full potential of the Bayesian search space remains unexplored.

To address these limitations, several extensions of the trading pipeline are planned for future research. In terms of prompt engineering and system efficiency, we intend to evaluate alternative prompt builders and investigate pre-generated prompt storage mechanisms for environments where prompt structures remain stable over time. Methodologically, future iterations will implement a robust and reproducible fold randomization scheme to replace fixed temporal cutoffs and obtain unbiased out-of-sample estimates. We will also investigate advanced reward shaping techniques to refine the asymmetric reward matrix and systematically assess the impact of removing or adding pre-processing pipelines, such as FinBERT and Chronos, to quantify their separate marginal contributions to final policy decisions. Additionally, we plan to implement a tripartite temporal memory module combining rolling buffers and self-reflection, alongside a dual-LLM architecture for semantic historical compression. Ultimately, automating model selection based on risk-adjusted metrics will ensure that future deployed agents strictly respect the maximum drawdown boundaries required by institutional standards.

5 Declaration on Generative AI

During the preparation of this work, the authors used large language models (including GPT-4 and Claude) for drafting assistance, grammatical revision, and LaTeX formatting. All generated content was subsequently reviewed, fact-checked, and edited by the authors, who take full responsibility for the final text and any remaining errors.

References

- [1] TheFinAI, *daily_news: Daily Financial News Dataset for FinMMEval*, https://huggingface.co/datasets/TheFinAI/daily_news, Accessed: 2025, 2025.
- [2] H. Yang, X.-Y. Liu, and C. D. Wang, “Fingpt: Open-source financial large language models,” *arXiv preprint arXiv:2306.06031*, 2023.
- [3] B. Zhang, H. Yang, and X.-Y. Liu, “Instruct-FinGPT: Financial Sentiment Analysis by Instruction Tuning of General-Purpose Large Language Models,” *arXiv preprint arXiv:2306.12659*, 2023. [Online]. Available: <https://arxiv.org/abs/2306.12659>
- [4] Y. Xiao, E. Sun, T. Chen, F. Wu, D. Luo, and W. Wang, “Trading-r1: Financial trading with llm reasoning via reinforcement learning,” *arXiv preprint*, 2025, Tauric Research.
- [5] D. Araci, “FinBERT: Financial Sentiment Analysis with Pre-trained Language Models,” *arXiv preprint arXiv:1908.10063*, 2019. [Online]. Available: <https://arxiv.org/abs/1908.10063>

- [6] A. F. Ansari et al., “Chronos: Learning the language of time series,” *Transactions on Machine Learning Research*, 2024. [Online]. Available: <https://openreview.net/forum?id=T08M3P4W1T>
- [7] W. Xu et al., “Finmultitime: A four-modal bilingual dataset for financial time-series analysis,” *arXiv preprint*, 2025.
- [8] L. Qian et al., “When agents trade: Live multi-market trading benchmark for llm agents,” *arXiv preprint*, 2025.
- [9] Y. Xiao, E. Sun, D. Luo, and W. Wang, “Tradingagents: Multi-agents llm financial trading framework,” *arXiv preprint arXiv:2412.20138*, 2024.
- [10] F. Tian, F. D. Salim, and H. Xue, “Tradinggroup: A multi-agent trading system with self-reflection and data-synthesis,” *arXiv preprint*, 2025.
- [11] C.-C. Lu, Y.-C. Chou, and T.-R. Chen, “Plgpt: A multi-agent llm workflow module for multi-modal financial information analysis,” *arXiv preprint*, 2025, Neurowatt AI.
- [12] E. J. Hu et al., “Lora: Low-rank adaptation of large language models,” in *International Conference on Learning Representations (ICLR)*, 2022.
- [13] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal policy optimization algorithms,” *arXiv preprint arXiv:1707.06347*, 2017.
- [14] G. Xiong et al., “Flag-trader: Fusion llm-agent with gradient-based reinforcement learning for financial trading,” *arXiv preprint arXiv:2502.11433*, 2025.
- [15] Z. Shao et al., “Deepseekmath: Pushing the limits of mathematical reasoning in open language models,” *arXiv preprint arXiv:2402.03300*, 2024.
- [16] DeepSeek-AI et al., “Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning,” *arXiv preprint arXiv:2501.12948*, 2025.